EX No:09

Date:

Apply Regularization & Dimension Reduction Methods using Boston Data

9.In this exercise, we will predict the number of applications received using the other variables in the College data set.

LOAD LIBRARY:

```
from ISLP import load_data
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.decomposition import PCA
from sklearn.cross_decomposition import PLSRegression
from sklearn.metrics import mean_squared_error
import statsmodels.api as sm
```

LOAD DATA:

```
College = load_data("College")

X = College.drop(columns=["Apps"])
y = College["Apps"]
X = pd.get_dummies(X, drop_first=True, dtype=int)
```

(a) Split the data set into a training set and a test set.

```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42)
print("Training set:", X_train.shape)
print("Test set:", X_test.shape)
```

OUTPUT:

```
Training set: (621, 17)
Test set: (156, 17)
```

(b) Fit a linear model using least squares on the training set, and report the test error obtained.

CODE:

```
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
mse_lr = mean_squared_error(y_test, y_pred_lr)
print("Linear Regression Test MSE:", mse_lr)
```

OUTPUT:

```
Linear Regression Test MSE: 1492443.3790390098
```

(c) Fit a ridge regression model on the training set, with λ chosen by cross-validation. Report the test error obtained.

CODE:

```
ridge = make_pipeline(
    StandardScaler(),
    RidgeCV(alphas=np.logspace(-6, 6, 13), cv=5) )
ridge.fit(X_train, y_train)
y_pred_ridge = ridge.predict(X_test)
ridge_mse = mean_squared_error(y_test, y_pred_ridge)
print("Best Ridge Lambda:", ridge.named_steps["ridgecv"].alpha_)
print("Ridge Test MSE:", ridge_mse)
```

OUTPUT:

```
Best Ridge Lambda: 0.1
Ridge Test MSE: 1490863.1190896018
```

(d) Fit a lasso model on the training set, with λ chosen by crossvalidation. Report the test error obtained, along with the number of non-zero coefficient estimates.

CODE:

```
lasso = make_pipeline(
    StandardScaler(),
    LassoCV(alphas=np.logspace(-6, 6, 50), cv=5, max_iter=100000))
lasso.fit(X_train, y_train)
y_pred_lasso = lasso.predict(X_test)
lasso_mse = mean_squared_error(y_test, y_pred_lasso)
print("Best Lasso Lambda:", lasso.named_steps["lassocv"].alpha_)
print("Lasso Test MSE:", lasso_mse)
nonzero = np.sum(lasso.named_steps["lassocv"].coef_ != 0)
print("Number of non-zero coefficients:", nonzero)
```

OUTPUT:

```
Best Lasso Lambda: 1e-06
Lasso Test MSE: 1492443.3748294376
Number of non-zero coefficients: 17
```

(e) Fit a PCR model on the training set, with M chosen by crossvalidation. Report the test error obtained, along with the value of M selected by cross-validation.

CODE:

```
from sklearn.model_selection import GridSearchCV
```

```
pcr = make_pipeline( StandardScaler(),PCA(),LinearRegression())
param_grid = {
   "pca__n_components": range(1, X_train.shape[1] + 1)
}
pcr_cv = GridSearchCV( pcr, param_grid, cv=5, scoring="neg_mean_squared_error")
pcr_cv.fit(X_train, y_train)
y_pred_pcr = pcr_cv.predict(X_test)
mse_pcr = mean_squared_error(y_test, y_pred_pcr)
print("Best M selected by CV:", pcr_cv.best_params_["pca__n_components"])
print("PCR Test MSE:", mse_pcr)
```

OUTPUT:

```
Best M selected by CV: 17
PCR Test MSE: 1492443.3790390256
```

(f) Fit a PLS model on the training set, with M chosen by crossvalidation. Report the test error obtained, along with the value of M selected by cross-validation.

CODE:

```
pls = make_pipeline(
   StandardScaler(),
   PLSRegression(scale=False)
)
param_grid = {
   "plsregression__n_components": range(1, X_train.shape[1] + 1)
}

pls_cv = GridSearchCV(
   pls,
   param_grid,
   cv=5,
   scoring="neg_mean_squared_error"
)

pls_cv.fit(X_train, y_train)
y_pred_pls = pls_cv.predict(X_test).ravel()
mse_pls = mean_squared_error(y_test, y_pred_pls)
print("Best M selected by CV:", pls_cv.best_params_["plsregression__n_components"])
print("PLS Test MSE:", mse_pls)
```

OUTPUT:

```
Best M selected by CV: 17
PLS Test MSE: 1492443.3790390247
```

(g) Comment on the results obtained. How accurately can we predict the number of college applications received? Is there much difference among the test errors resulting from these five approaches?

CODE:

```
results = pd.DataFrame({
   "Method": [ "Linear Regression","Ridge", "Lasso","PCR","PLS" ],
   "Test MSE": [mse_lr,ridge_mse,lasso_mse,mse_pcr,mse_pls ] })
results.sort_values("Test MSE")
```

OUTPUT:

	Method	Test MSE
1	Ridge	1.490863e+06
2	Lasso	1.492443e+06
0	Linear Regression	1.492443e+06
4	PLS	1.492443e+06
3	PCR	1.492443e+06

CODE:

```
best_model = results.loc[results["Test MSE"].idxmin()]

print("Best Model:", best  model["Method"])
print("Lowest Test MSE:", best_model["Test MSE"])
```

OUTPUT:

```
Best Model: Ridge
Lowest Test MSE: 1490863.1190896018
```

INTERPRETATION:

- (a) The 777 colleges are split 80/20 into 621 training and 156 test observations, using 17 predictors (Private is dummy-coded).
- (b) to (f) Test MSE: linear regression 1,492,443; ridge 1,490,863 (lambda = 0.1); lasso 1,492,443 (lambda = 1e-6, all 17 coefficients non-zero); PCR 1,492,443 (M = 17); PLS 1,492,443 (M = 17).
- Lasso, PCR and PLS all select essentially no shrinkage or reduction (all 17 coefficients or components kept), so they reproduce the least-squares fit exactly. Ridge, with a very small lambda, is lower by about 1,580 (roughly 0.1%), which is negligible.
- (g) The best model is Ridge (lowest test MSE 1,490,863), but there is no meaningful difference among the five approaches. With only 17 predictors and 621 training rows there is little overfitting, so regularisation and dimension reduction add almost nothing.
- Accuracy: the test RMSE is about 1,220 applications, compared with a mean of about 3,002 and a standard deviation of about 3,870 for Apps. That corresponds to roughly 90% of the variance explained (a rough estimate using the full-data standard deviation), so the number of applications can be predicted reasonably well but with errors of about 1,200 applications.